



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Proyecto final

Simulación de Procesos Distribuidos usando Docker, gRPC y
Relojes de Lamport.

Profesora:

M.I. Elba Karen Saenz García

Fecha de Entrega:

10 de Junio del 2026

Índice

1	Objetivo	1
2	Planteamiento del escenario:	1
3	Diagrama del escenario:	2
4	Documentacion de codigo:	3
4.1	Codigo proto:	3
4.2	Codigos python:	3
4.3	Códigos docker:	4
5	Secuencia de comandos para la ejecución:	7
5.1	Preparación del entorno	7
5.2	Secuencia de comandos para el escenario esto, para la terminal 2	7
5.3	Revision de logs:	10
6	Resultados:	10
6.1	Capturas de los resultados:	10
6.2	Bitacoras:	10
7	Conclusion:	10
8	Link de video:	10
	Referencias	10



1. Objetivo

Comprender cómo se mantiene el orden lógico de eventos en sistemas distribuidos mediante el uso de relojes vectoriales.

2. Planteamiento del escenario:

Para el escenario, se tomo base las aplicaciones de reparto de comida donde en este punto tomamos como referencia una arquitectura de microservicios, esto mas precisamente cuando un cliente hace una orden, en nuestro caso tomamos como referencia al restaurante Carls jr, donde tomamos como pedido la hamburguesa Famosa star, esto lo tomamos desde que el usuario selecciona la hamburguesa hasta que el sistema manda la notificación final de entrega, ahora bien, tomamos a los procesos como las responsabilidades.

Para esto, todo comienza en el Proceso 1 que es como tal la Aplicación Cliente, este proceso simula la UI de usuario propiamente, donde como tal el evento interno es agregar la comida al carrito de compras y el monto a pagar, despues de esto, una vez que la orden está lista, el Proceso 1 envía una solicitud de cobro mediante un mensaje punto a punto al Proceso 2 que este sera nuestro movimiento de Pago (donde se verifica si la tarjeta es verdadera y tiene fondos), transmitiendo su reloj vectorial inicial para emepzar la sincronización temporal del sistema.

Despues de esto, al recibir la solicitud, el Proceso 2 actualiza su reloj local y ejecuta un evento interno que es el de validar los datos de la tarjeta y procesar la propia transeaccion, despues tras aprobar el pago, este proceso se comunica con el Proceso 3 que en este punto lo estamos simulando como si fuera la aplicacion que se tiene en la cocina del Restaurante, enviándole la confirmación de fondos aprobados y los detalles de la misma orden, en este caso que tipo de hambrugesa va a desear el cliente, esto lo que hace es que garantiza un orden lógico, donde como tal el restaurante no comenzará a cocinar sin que el pago haya sido exitoso.

Posteriormente, el Proceso 3 registra un evento interno simulando la preparación y el empaquetado de la orden, de hecho una vez que el paquete está listo en el mostrador, la cocina enviara una notificación directa al Proceso 4 que es ya directamente la aplicación del Repartidor. Este cuarto proceso recibe la instrucción, sincroniza su vector, y lanza su propio evento interno para calcular la ruta óptima de entrega mediante el GPS hacia el domicilio del cliente. Al iniciar el viaje, el repartidor actualiza el estado de la orden enviando un mensaje al último proceso.

Ya por ultimo, tenemos al Proceso 5 que es el servidor de notificaciones, donde en este al recibir el mensaje del repartidor, este ejecuta un evento interno para registrar el Tiempo Estimado de Llegada que es el ETA, y para acabar el ciclo de la orden, el Proceso 5 emite un Broadcast hacia los cuatro procesos restantes P1, P2, P3 y P4 indicando que la orden ha sido completada y entregada de forma exitosa.



Este ultimo paso es importante, debido a que es el punto donde el sistema logra sincronizar completamente todos los estados haciendo que los cinco relojes vectoriales sean los mismos de manera simultánea.

Servicios gRPC

1. **Enviar Mensaje:** Recibe una estructura ReqEnvio con los identificadores de origen, destino y de mensaje, esto de forma estructurada como una cadena, despues de esto se incrementa la posición en el Reloj Vectorial del emisor y coloca la línea de envío en la bitácora.
2. **Recibir Mensaje:** Recibe una estructura ReqRecepcion que contiene el proceso de origen, el mensaje y el arreglo del vector, despues de esto actualiza el reloj local calculando los valores máximos, esto por de forma que lo hace componente a componente, e incrementando en 1, por ultimo registra el evento de recepción y devuelve un mensaje de confirmación con el vector del reloj con el tiempo actual.
3. **Evento Interno:** Recibe una solicitud ReqInterno con la descripción de una tarea (armado de carrito, validación de pago, preparación en cocina o cálculo de ruta GPS), despues este incrementa la posición en su reloj lógico vectorial y pone de forma síncrona la entrada en el archivo de registro.
4. **Broadcast:** Recibe una petición de difusión masiva, est emimso incrementa el reloj vectorial del emisor en 1 y atiende de manera por igual al mismo tiempo el estado actual junto con el mensaje a todos los demás procesos para sincronizar el estado de la orden.

3. Diagrama del escenario:

Descripcion del reloj vectorial:



Diagrama:

4. Documentacion de codigo:

4.1. Codigo proto:

4.2. Codigos python:



4.3. Códigos docker:

Dockerfile:

El archivo Dockerfile lo que hace es que define las instrucciones de como se ira contruyendo la imagen base de los contenedores.

Para esto la primer alinea, lo que ahce es que inicia una imagen optimizada de Python python:3.11-slim, depsues de esto, establece /app como el directorio de trabajo principal dentro del entorno virtual.

Despues de esto, transfiere el archivo de requerimientos e instala las dependencias mediante pip install, utilizando la bandera --no-cache-dir, esto se hace mas que nada para evitar almacenar archivos temporales y reduce el peso del contenedor final, despues de esto, copia los directorios de código que en neustro los tenemos por carpetas separadas, que son proto y src, y configura la variable de entorno PYTHONPATH esto se hace para que el intérprete reconozca los módulos importados.

No obstante en la 13 linea, tenemos lo mas importante, que es directaemtne el lanzamiento de el servidor gRPC, esto por medio de la compilacion inicial del archivo proto, este lo que hara es que procesa el archivo procesos.proto para generar las clases cliente y servidor en Python.

Finalmente, declaramos el uso del puerto 50051 para permitir la comunicacion de el gRPC y definemos como comando por defecto la ejecución del servidor principal src/process.py.

Listing 1: Código dockerfile

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt ./
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY proto ./proto
9 COPY src ./src
10
11 ENV PYTHONPATH=/app/src
12
13 RUN python -m grpc_tools.protoc -I ./proto \
14     --python_out=./src \
15     --grpc_python_out=./src \
16     ./proto/procesos.proto
17
18 EXPOSE 50051
19
20 CMD ["python", "src/process.py"]
```



Docker Compose:

El archivo docker-compose es el que se encarga de organizar directamente toda la estructura completa del sistema, definiendo desde la red aislada y un total de seis servicios, donde en este caso son cinco procesos de procesamiento p1 a p5 y un proceso o nodo que es el cliente.

Para cada uno de los procesos servidores, el código lo que hace, es que indica que la imagen debe construirse a partir del directorio actual y le asigna un nombre de contenedor, esto por medio del comando `environment`, después de esto se colocan ya directamente las variables de entorno para la lógica del código en Python, que en este caso son las de los requisitos del proyecto, tales como el identificador único del proceso `PROCESS_ID`, el mapa de la red indicando las rutas de todos los nodos y la ruta de destino para almacenar directamente las bitácoras.

Para asegurar que los archivos generados no se pierdan al apagar los contenedores, se utiliza `columns` directamente, que lo que hace es que, mapea un directorio local hacia dentro del propio contenedor.

No obstante, el servicio cliente es casi igual, pero sobrescribe el comando de inicio por defecto con `"sleep."infinity"`, esto es por que lo que queremos es que mantenga en un estado de reposo para poder seguir mintiendo comandos de prueba, que son los **docker exec**.

Finalmente, en la parte inferior del archivo, se define la red_privada bajo un formato de tipo puente con una subred que en este caso es parecido al usado directamente en clase en los ejercicios, siendo para este proyecto el 172.21.0.0/16, lo cual permite usar direcciones estáticas para cada contenedor.

Listing 2: Código docker compose

```
1 services:
2   p1:
3     build: .
4     container_name: p1
5     environment:
6       PROCESS_ID: P1
7       PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
8       LOG_PATH: /app/logs/p1.log
9     volumes:
10      - ./logs:/app/logs
11     networks:
12       red_privada:
13         ipv4_address: 172.21.0.2
14
15   p2:
16     build: .
17     container_name: p2
18     environment:
19       PROCESS_ID: P2
20       PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
21       LOG_PATH: /app/logs/p2.log
```



```
22 volumes:
23   - ./logs:/app/logs
24 networks:
25   red_privada:
26     ipv4_address: 172.21.0.3
27
28 p3:
29   build: .
30   container_name: p3
31   environment:
32     PROCESS_ID: P3
33     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
34     LOG_PATH: /app/logs/p3.log
35   volumes:
36     - ./logs:/app/logs
37   networks:
38     red_privada:
39       ipv4_address: 172.21.0.4
40
41 p4:
42   build: .
43   container_name: p4
44   environment:
45     PROCESS_ID: P4
46     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
47     LOG_PATH: /app/logs/p4.log
48   volumes:
49     - ./logs:/app/logs
50   networks:
51     red_privada:
52       ipv4_address: 172.21.0.5
53
54 p5:
55   build: .
56   container_name: p5
57   environment:
58     PROCESS_ID: P5
59     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
60     LOG_PATH: /app/logs/p5.log
61   volumes:
62     - ./logs:/app/logs
63   networks:
64     red_privada:
65       ipv4_address: 172.21.0.6
66
67 cliente:
68   build: .
69   container_name: cliente
70   command: ["sleep", "infinity"]
71   environment:
72     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
73   volumes:
```




```
74 - ./logs:/app/logs
75 networks:
76   red_privada:
77     ipv4_address: 172.21.0.10
78
79 networks:
80   red_privada:
81     driver: bridge
82     ipam:
83       config:
84         - subnet: 172.21.0.0/16
```

5. Secuencia de comandos para la ejecución:

5.1. Preparación del entorno

Levantar los contenedores: Se levanta todo el sistema la mediante, el siguiente comando, donde la bandera -d libera la terminal.

```
docker compose up -d -build
```

Ver los logs en vivo (Terminal 1):

En dado caso, de que si quiera ver los logs, en vivo se hara uso de dos terminal, no obstante como bien sabemos los logs se guardan en una carpeta llamada logs, y se puede ver con el uso del comando cat, regresando a la terminal 1, se dejara con este comando activo. Esto permite observar cómo los contenedores reaccionan en tiempo real.

```
docker compose logs -f
```

5.2. Secuencia de comandos para el escenario esto, para la terminal 2

1. P1 (App Cliente) – Evento interno: crear el carrito

Inicia el evento interno en P1 simulando que la app del cliente crea la orden.

P1 incrementa su propia posición en el reloj vectorial, pasando de $[0,0,0,0,0]$ a $[1,0,0,0,0]$.

en este punto no hay comunicación con otros procesos.

Comando:

```
docker exec -it cliente python src/cli.py --destino P1 interno --desc "armar carrito pedido"
```



2. P1 → P2 - solicitud de cobro

P1 envía un mensaje gRPC punto a punto a P2 con los datos de la orden.

Al enviarlo, P1 incrementa su reloj nuevamente y envía su vector al mensaje.

Al recibirlo, P2 compara los vectores numero a numero del arreglo, toma los máximos e incrementa su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P1 enviar --receptor P2 --msg '{"id":"ORD-77X",  
"total":250, "req":"cobro"}
```

3. P2 (Verficiacion de Pago)

P2 ejecuta un evento interno simulando que se verifica si tarjeta de crédito tiene fondos y es real.

Solo incrementa su posición en el reloj vectorial.

Comando:

```
docker exec -it cliente python src/cli.py --destino P2 interno --desc "validar cobro tarjeta"
```

4. P2 → P3 - Confirmacion al restaurante

Con el pago aprobado, P2 le envía los detalles de la orden a P3 vía gRPC.

P3 recibe el vector de P2, aplica lo mismo de revisar los máximos e incrementa su propia posición.

Su reloj ahora esta sincornizado con los eventos previos de P1 y P2.

Comando:

```
docker exec -it cliente python src/cli.py --destino P2 enviar --receptor P3 --msg '{"id":"ORD-  
77X", "pago":"ok", "item":"Super Star"}'
```

5. P3 (Restaurante)

P3 ejecuta un evento interno que lo que hace es que simula la preparación y que se empaqueta la hamburguesa.

Su reloj vectorial avanza en su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P3 interno --desc "preparar comida"
```



6. P3 → P4 - Aviso al repartidor

P3 envía un mensaje gRPC a P4 indicando que el paquete está listo para recolección.

P4 combina directamente el vector recibido con el suyo, sincronizando su reloj con todos los eventos acumulados.

Comando:

```
docker exec -it cliente python src/cli.py --destino P3 enviar --receptor P4 --msg '{"id":"ORD-77X", "paquete":"listo"}
```

7. P4 (Repartidor) - Evento interno: calcular ruta GPS

P4 realiza un evento interno que simula el cálculo de la ruta óptima hacia el destino del cliente mediante GPS.

Solo incrementa su posición en el reloj vectorial.

Comando:

```
docker exec -it cliente python src/cli.py --destino P4 interno --desc "calcular ruta gps"
```

8. P4 → P5 – Notificación de inicio de viaje

P4 le comunica a P5 que es el servidor de notificaciones que el repartidor ya está en camino.

P5 recibe el vector con el historial completo de todos los procesos y lo combinad directamente con el suyo para preparar el broadcast.

Comando:

```
docker exec -it cliente python src/cli.py --destino P4 enviar --receptor P5 --msg '{"id":"ORD-77X", "repartidor":"va", "eta":"15m"}
```

9. P5 (Servidor de Notificaciones)

P5 ejecuta un evento interno para calcular y registrar el tiempo estimado de llegada antes de mandar el broadcast.

Su reloj vectorial avanza en su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P5 interno --desc "enviar notificacion"
```



10. P5 – Broadcast final a todos los nodos

P5 envía a todos al mismo tiempo el estado final de la orden a P1, P2, P3 y P4 mediante una difusión gRPC.

Cada proceso receptor compara el vector actualizado de P5 con el suyo y termina con los tiempos actualizados de forma correcta todos.

Comando:

```
docker exec -it cliente python src/cli.py --destino P5 difusion --msg '{"id":"ORD-77X", "fin":"entregado", "eta":"15m"}'
```

5.3. Revision de logs:

Para revisar el archivo de bitácora generado por cada nodo al finalizar la secuencia (se muestra el ejemplo para P1, pero aplica para P1 a P5):

```
docker exec -it p1 cat /app/logs/p1.log
```

Para detener los contenedores, esto solo se hace después de matar el proceso del comando de logs, esto baja todos los servicios de docker creados:

```
docker compose down
```

6. Resultados:

6.1. Capturas de los resultados:

6.2. Bitácoras:

7. Conclusion:

8. Link de video:

Referencias

- Docker. (2026). *Docker Documentation*. Consultado el 28 de mayo de 2026, desde <https://docs.docker.com/>
- gRPC. (2026). *gRPC Documentation*. Consultado el 28 de mayo de 2026, desde <https://grpc.io/docs/>
- Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>